2024-2025 学年秋季学期

《计算机网络》

评分：_____

学　　院：计算机科学与工程学院
专　　业：计算机科学与技术
班　　级：计算机 2204 班
姓　　名：李俊洁
学　　号：20225443
任课教师：刘思宇、贾杰

1 引言

Socket 编程是一种通过计算机网络实现客户端与服务器端之间信息交互的技术。它广泛应用于分布式系统、即时通讯、网络服务等领域。本文档介绍了利用 Python 实现的客户端与服务器之间的信息交互应用，旨在展示如何使用 Socket 编程构建基础的通信系统，并提供相关代码和分析说明。

在本实验中，使用 Python 的标准库构建了一个简单的 TCP 客户端和服务器。服务器能够处理多个客户端的连接请求，并实现消息广播功能，而客户端则可以与服务器进行双向通信。本文档详细描述了系统的设计、实现和运行流程。

2 系统描述

本节将详细介绍客户端与服务器的设计目标、实现技术和具体功能。

2.1 服务器实现

服务器端采用 Python 实现，使用 Socket 模块处理 TCP 连接，能够支持多个客户端的连接并实现广播功能。

服务器端的主要功能包括：

- 监听指定的 IP 和端口，接受多个客户端的连接。
- 处理来自客户端的消息，并将其广播给其他所有已连接的客户端。
- 向发送消息的客户端提供反馈，确认消息已被接收。

以下是服务器端的代码片段：

```python
import socket
import threading

# 定义服务器的IP和端口
HOST = '127.0.0.1'  # 本地IP地址
PORT = 65432        # 服务器的端口

# 存储客户端连接的字典 (IP地址作为标识)
clients = {}

# 处理每个客户端连接的函数
def handle_client(conn, addr):
    print(f"已连接客户端: {addr}")

    # 将新客户端添加到 clients 字典中
    clients[addr] = conn

    while True:
        try:
            # 接收客户端发送的数据
            data = conn.recv(1024)
            if not data:
                break  # 客户端关闭连接

            print(f"收到来自客户端 {addr} 的消息: {data.decode()}")

            # 向所有客户端广播消息
            broadcast(data.decode(), addr)
```

```
            # 向客户端发送响应
            response = f"服务器收到: {data.decode()}"
            conn.sendall(response.encode())
        except:
            break

    # 客户端断开连接
    print(f"客户端 {addr} 断开连接")
    conn.close()
    del clients[addr]  # 从客户端列表中移除断开的客户端

# 向所有客户端广播消息
def broadcast(message, client_addr=None):
    for addr, conn in clients.items():
        if addr != client_addr:  # 排除发送者自己
            try:
                conn.sendall(f"广播消息: {message}".encode())
            except:
                # 发送失败，移除该客户端
                print(f"无法向 {addr} 发送消息，客户端可能已断开")
                del clients[addr]

# 创建服务器Socket
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server_socket.bind((HOST, PORT))
server_socket.listen(5)

print(f"服务器正在监听 {HOST}:{PORT} ...")

# 不断接受客户端连接并创建线程处理
while True:
    conn, addr = server_socket.accept()
    # 为每个客户端连接创建一个新的线程
    client_thread = threading.Thread(target=handle_client, args=(conn, addr))
    client_thread.start()
```

Listing 1: 服务器端代码片段

2.2 客户端实现

客户端同样使用 Python 实现，通过 Socket 与服务器进行连接，并能够发送和接收消息。客户端的主要功能包括：

- 连接到指定服务器的 IP 地址和端口。
- 向服务器发送消息，并接收服务器的响应。
- 接收来自服务器的广播消息。

以下是客户端的代码片段：

```
import socket
import threading

# 定义服务器的IP和端口
HOST = '127.0.0.1'  # 服务器的IP地址
PORT = 65432        # 服务器的端口

# 创建一个TCP/IP套接字
```

```python
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# 连接到服务器
client_socket.connect((HOST, PORT))

# 接收服务器推送的消息
def receive_messages():
    while True:
        try:
            data = client_socket.recv(1024)
            if data:
                print(f"收到服务器消息: {data.decode()}")
        except:
            print("与服务器的连接断开")
            break

# 启动接收消息的线程
threading.Thread(target=receive_messages, daemon=True).start()

# 与服务器交互
print("已连接到服务器！")
while True:
    message = input("请输入要发送给服务器的消息 (输入 'exit' 退出): \n")

    if message.lower() == 'exit':
        print("断开连接...")
        client_socket.close()
        break

    # 向服务器发送消息
    client_socket.sendall(message.encode())

    # 接收服务器的响应
    data = client_socket.recv(1024)
    print(f"来自服务器的响应: {data.decode()}")
```

Listing 2: 客户端代码片段

3 方法

3.1 使用的技术和库

客户端和服务器程序均使用 Python 实现，并使用以下库：

- `socket`：用于创建套接字，处理客户端与服务器之间的网络通信。
- `threading`：用于支持多线程，以实现服务器端同时处理多个客户端的连接，以及客户端接收服务器推送的消息。

3.2 工作流程

系统的工作流程如下：

1. 服务器启动，监听指定的 IP 和端口，等待客户端连接。
2. 客户端连接服务器，并能够向服务器发送消息。
3. 服务器接收到来自客户端的消息后，将其广播给其他所有已连接的客户端。

4. 客户端可以接收来自服务器的广播消息，实现基本的信息交互功能。

4 使用场景

本次实验的目标受众是对计算机网络和分布式系统感兴趣的学生和开发者。通过本实验，可以学习到：

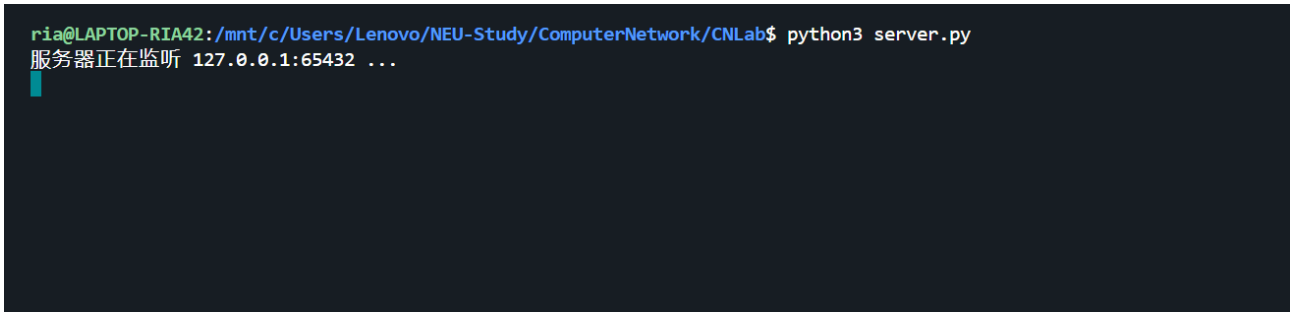
- 如何使用 Socket 编程实现客户端与服务器之间的通信。
- 如何处理并发连接，支持多客户端的接入。
- 如何在服务器与多个客户端之间实现消息的广播和响应。

4.1 收获

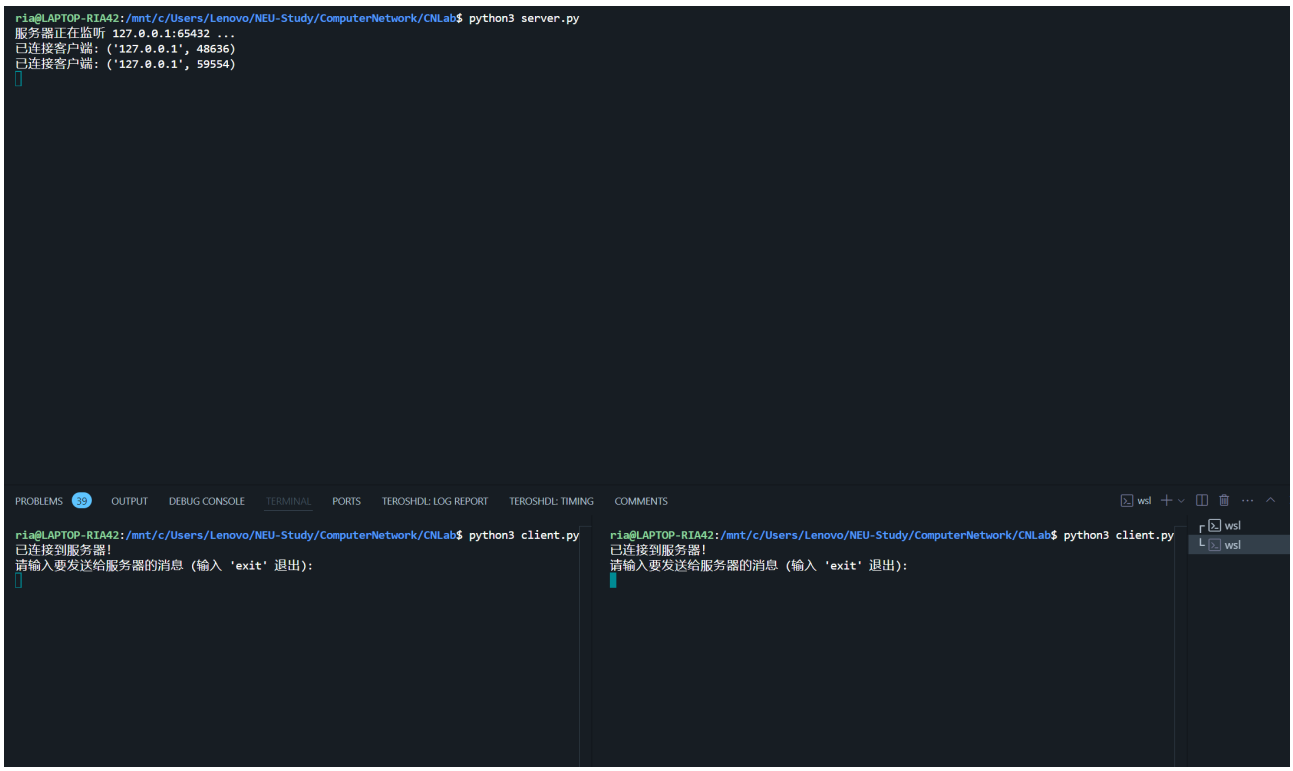
- 实现一个简单的聊天系统，支持多个客户端与服务器的消息交流。
- 学习网络编程的基础知识，理解 Socket 通信的原理和实现方式。
- 为后续实现更复杂的分布式系统奠定基础，例如实时协作工具或在线游戏等。

5 演示

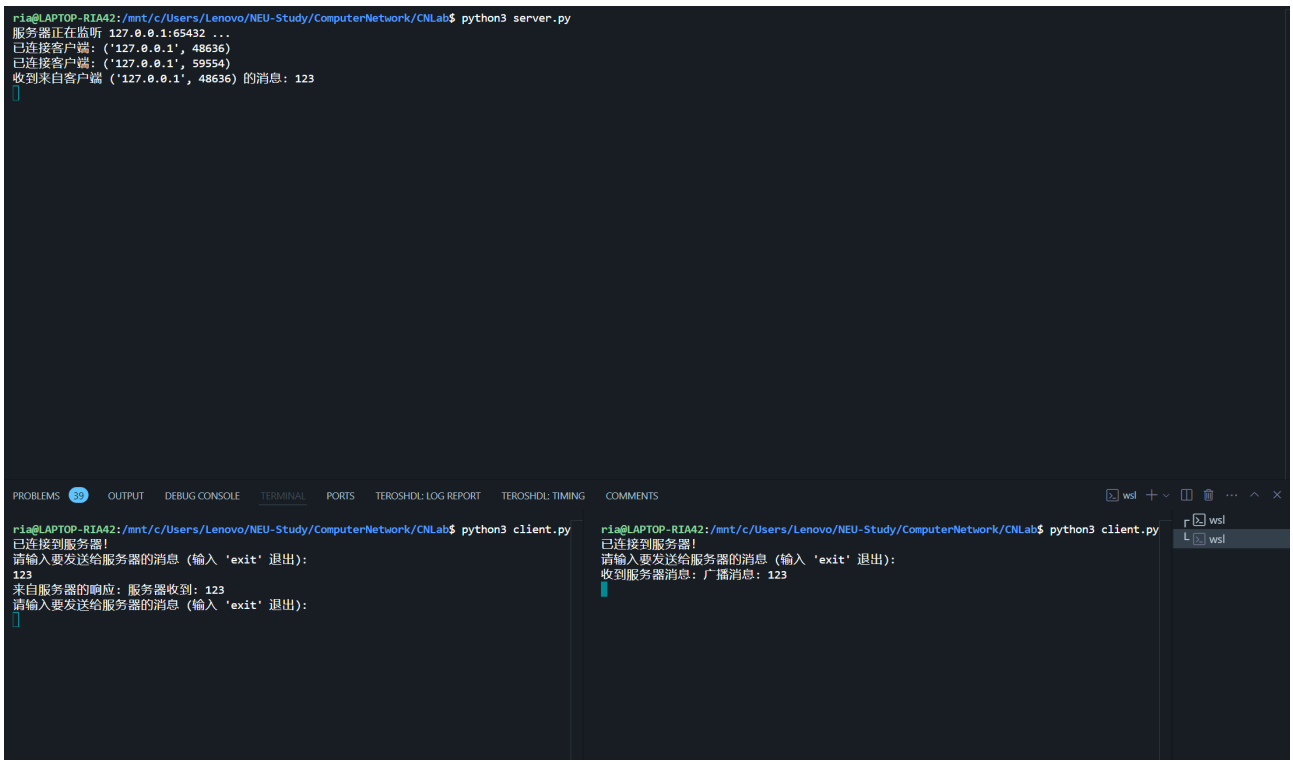
首先打开服务器终端：



随后打开客户端终端，其中客户终端可以多开，服务器将自动分配端口：



客户端发送消息时，能够接收到服务器的响应，并且消息将广播发送：



服务器和客户端能够实现各种复杂的消息交互，并且在服务器能够实时监听，可以发现到客户端的连接和断开：

6 实验结果与分析

在本实验中，通过客户端与服务器的交互，我们验证了 Socket 编程的可行性和实用性。具体结果如下：

- 客户端成功连接服务器，能够发送消息并接收服务器的响应。
- 服务器能够接受多个客户端的连接，并能够处理来自不同客户端的消息。
- 广播功能有效，所有连接的客户端都能收到来自其他客户端的消息。

7 结论

本文档介绍了使用 Python 实现的 Socket 编程实验，包括客户端和服务器之间的信息交互。通过该实验，展示了如何使用 Socket 进行基础的网络通信，并实现了多客户端的连接与消息广播功能。本实验有助于理解计算机网络通信的基本原理，为学习和开发复杂的网络应用打下基础。

8 未来工作

未来可以考虑以下改进：

- 添加用户身份验证机制，确保只有授权用户才能连接到服务器。
- 优化服务器性能，以支持更多并发连接，提升系统稳定性。
- 使用加密技术（如 TLS/SSL）来确保客户端和服务器之间数据传输的安全性。

姓名: 李俊洁
班级: 计算机 2204
学号: 20225443
课程: 计算机网络
日期: November 6, 2024
制作: LaTeX
感谢您的批阅！

References

[1] Python Socket 文档：https://docs.python.org/3/library/socket.html

[2] Threading 文档：https://docs.python.org/3/library/threading.html